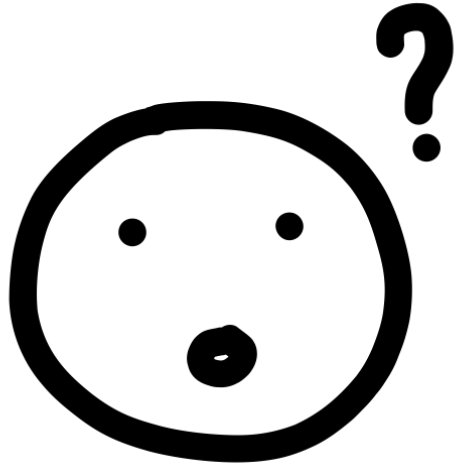
Objects and data types

Session overview

1. Objects and object assignment
2. Vectors
3. Data types (and converting between them)
4. Missing values and factors
5. Lists and matrices



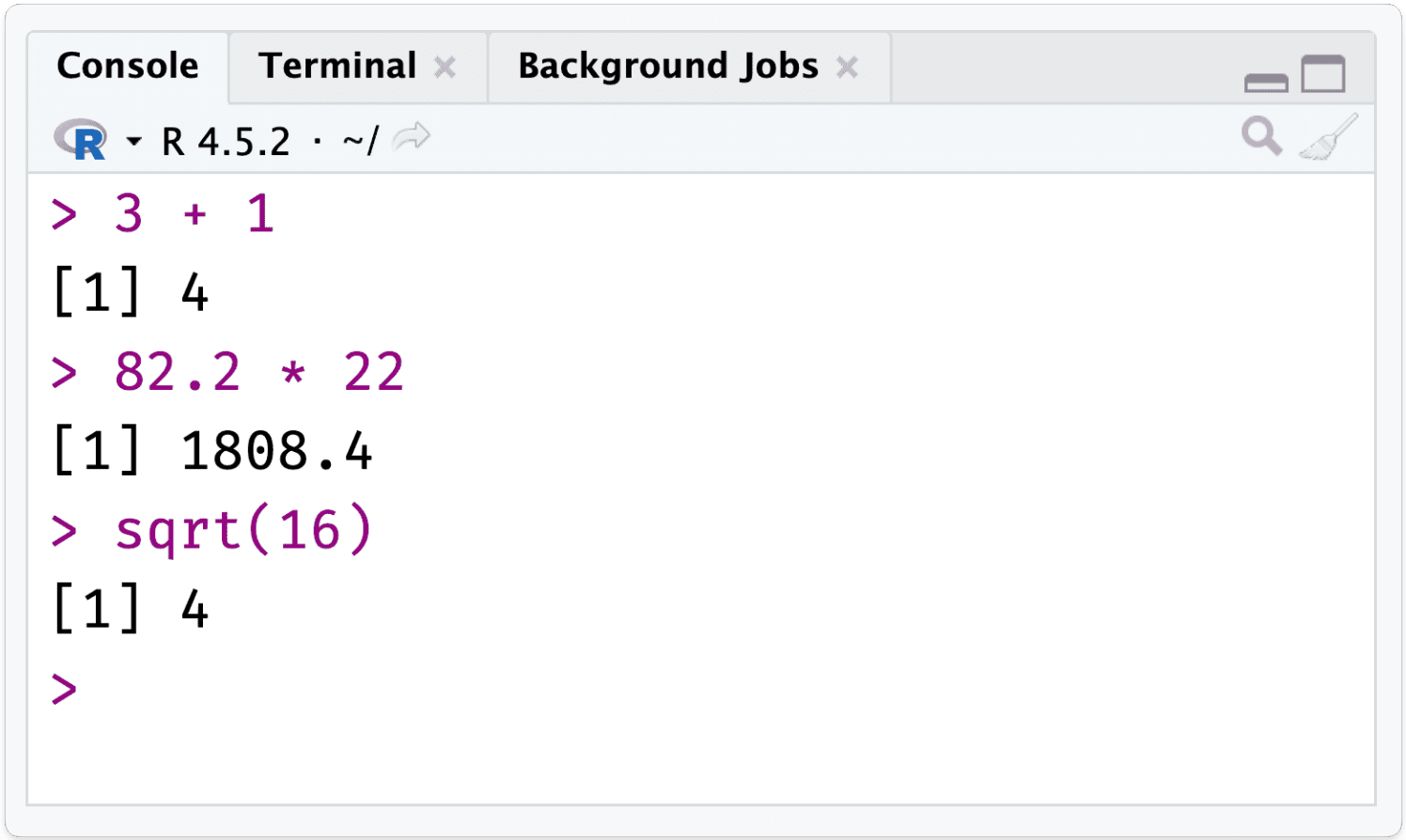


These topics are fairly abstract.

It will make sense later.

Objects and object assignment

The R console can be used interactively, like a calculator.



For more complex operations, we need to store objects in memory.

- Use the result of one calculation as the input for another.
- Load data and analyse it.

We need a way to name and refer to objects.

The assignment arrow

We can store something as an object by giving it a name:

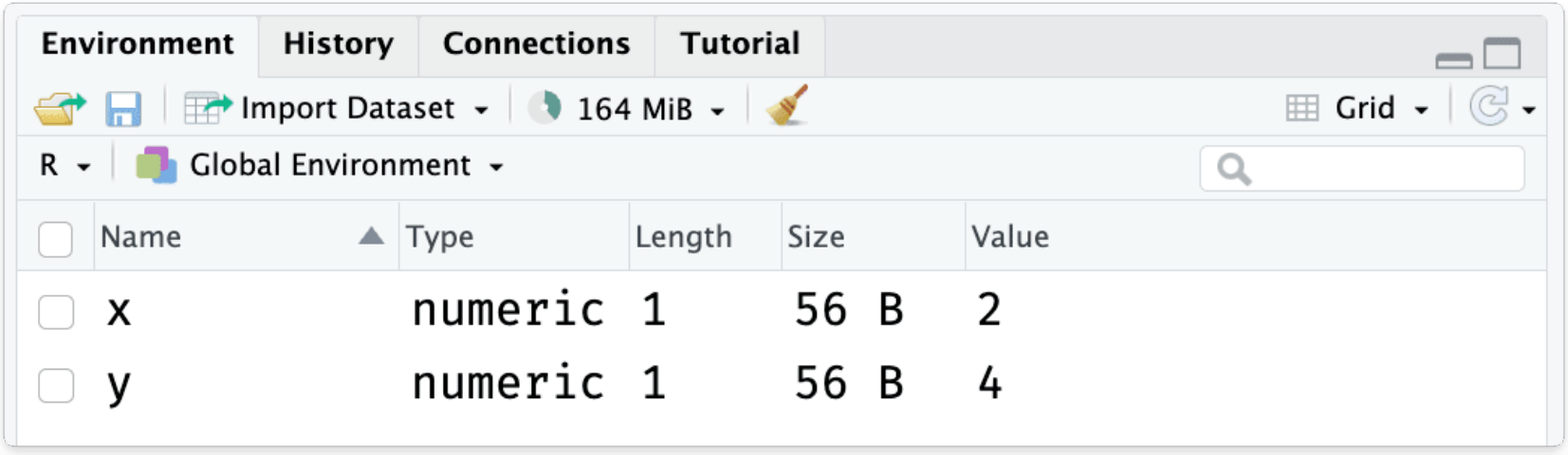
```
x <- 2
y <- 4
```

We can then use stored objects in subsequent calculations:

```
z <- x * y
```

⌨ Have a go now.

Created objects live in the global environment. We can view them with `ls()` or in the 'Environment' pane:



They stay there until we restart R.

We can remove them with `rm()` or via the 'Environment' pane in RStudio.

In R, almost everything you work with is an object.

- Numbers, strings, vectors, lists, and data frames are objects.
- Functions are objects.
- Even model fits and plots are objects you can store and reuse.

Vectors

Vectors are the simplest object

A vector is a one-dimensional object containing values of a single type.



We can create them with `c()`.

```
x1 <- 5
x2 <- c(5, 7)
x3 <- c("A", "B", "C", "D")
```

Objects of length 1, 2, or 3 are still vectors.

Creating vectors

```r
# By hand
one_to_five <- c(1, 2, 3, 4, 5)

# Using the 'seq' function
lazy <- seq(from = 1, to = 5, by = 1)

# Same, but without naming the arguments
lazier <- seq(1, 5, 1)

# Using ':'
laziest <- 1:5
```

Data types

Every object in R has a type

```
x <- 1
y <- "A"
z <- TRUE
```

We can use `typeof` and `str` to inspect an object's type and structure:

```
> typeof(x)
[1] "double"
> typeof(y)
[1] "character"
> typeof(z)
[1] "logical"
> str(x)
 num 1
```

You'll come across several data types in R:

- Numeric (e.g., `1.0`, `2e12`)
- Character (e.g., `"like this"`)
- Logical (e.g., **`TRUE`**, **`FALSE`**)
- Factor
- Missing values (e.g., **`NA`**)
- Dates, times, and intervals
- ...

R has standard ways to create each type and convert between them.

Whole numbers can also be stored as integers (`1L`, or `as.integer(1)`). The difference rarely matters in practice.

Characters (or 'strings')

```
> first <- "Joe"
> last <- "Bloggs"
> age <- "40"

> is.character(first)
[1] TRUE

> paste(first, last)
[1] "Joe Bloggs"

> age + 10
Error in age + 10 : non-numeric argument...
> age <- as.numeric(age)
> age + 10
[1] 50
```

Logical (or 'boolean') values

```
> 5 > 4
[1] TRUE

> "Joe" == "Bloggs"
[1] FALSE

> "Joe" == "Joe"
[1] TRUE

> typeof(TRUE)
[1] "logical"

> str(TRUE)
 logi TRUE
```

```
> !(TRUE)
[1] FALSE

> TRUE & FALSE
[1] FALSE

> TRUE | FALSE
[1] TRUE

> any(TRUE, FALSE, FALSE)
[1] TRUE

> all(TRUE, FALSE, FALSE)
[1] FALSE
```

Type coercion (when types mix)

When you mix types in a vector, R coerces them to a single common type.

```
> c(1, "2")
[1] "1" "2"
> c(TRUE, 2)
[1] 1 2
```

Watch out: one character (or `NA`) can silently change the whole vector.

```
> typeof(c(TRUE, 2))
[1] "double"
> typeof(c(1L, 2.5))
[1] "double"
> typeof(c(1, "2"))
[1] "character"
```

Missing values and factors

Missing values are represented with `NA`

We can check for missing values:

```
> is.na(NA)
[1] TRUE
```

Do not confuse `NA` with `NaN`.

> *NaN, not a number, a numeric value representing an undefined or unrepresentable value.*

```
> 0/0
[1] NaN
> is.nan(0/0)
[1] TRUE
```

```
> is.nan(1)
[1] FALSE
> is.na(NaN)
[1] TRUE
```

Binary values

We can represent binary values as logical (`TRUE`/`FALSE`) or numeric (`0`/`1`).

For example, create a new binary variable with a conditional expression:

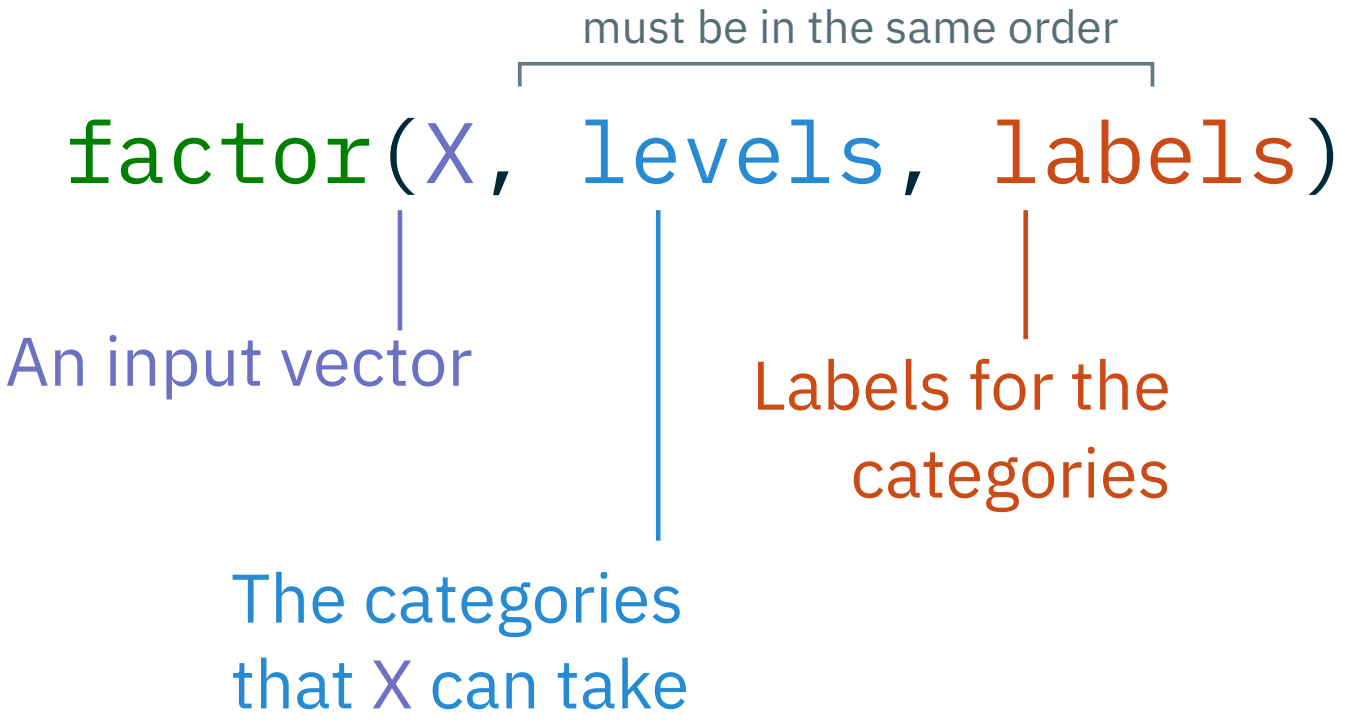
```
> mtcars$ineff <- mtcars$mpg < 15
> mtcars$ineff
[1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[7] FALSE FALSE  TRUE  TRUE   TRUE FALSE
```

We can then convert it to numeric (`FALSE` becomes `0`; `TRUE` becomes `1`).

```
> as.numeric(mtcars$ineff)
 [1] 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 1 [...]
```

Factors

Factors represent categorical data (e.g., nominal or ordinal) in terms of a numeric value and an associated label.



If you're familiar with Stata, this is similar to 'values' and 'value labels'.

We'll return to factors in Session 4, for plotting.

Dates

R has a Date type. We can convert text to dates:

```
> start <- as.Date("2026-11-02")
> class(start)
[1] "Date"
```

And then do calculations with them:

```
> start + 7
[1] "2026-11-09"
> as.Date("2026-11-30") - start
Time difference of 28 days
```

We'll cover dates and times properly in Session 4.

Lists and matrices

Matrices

A matrix is a rectangular array of data:

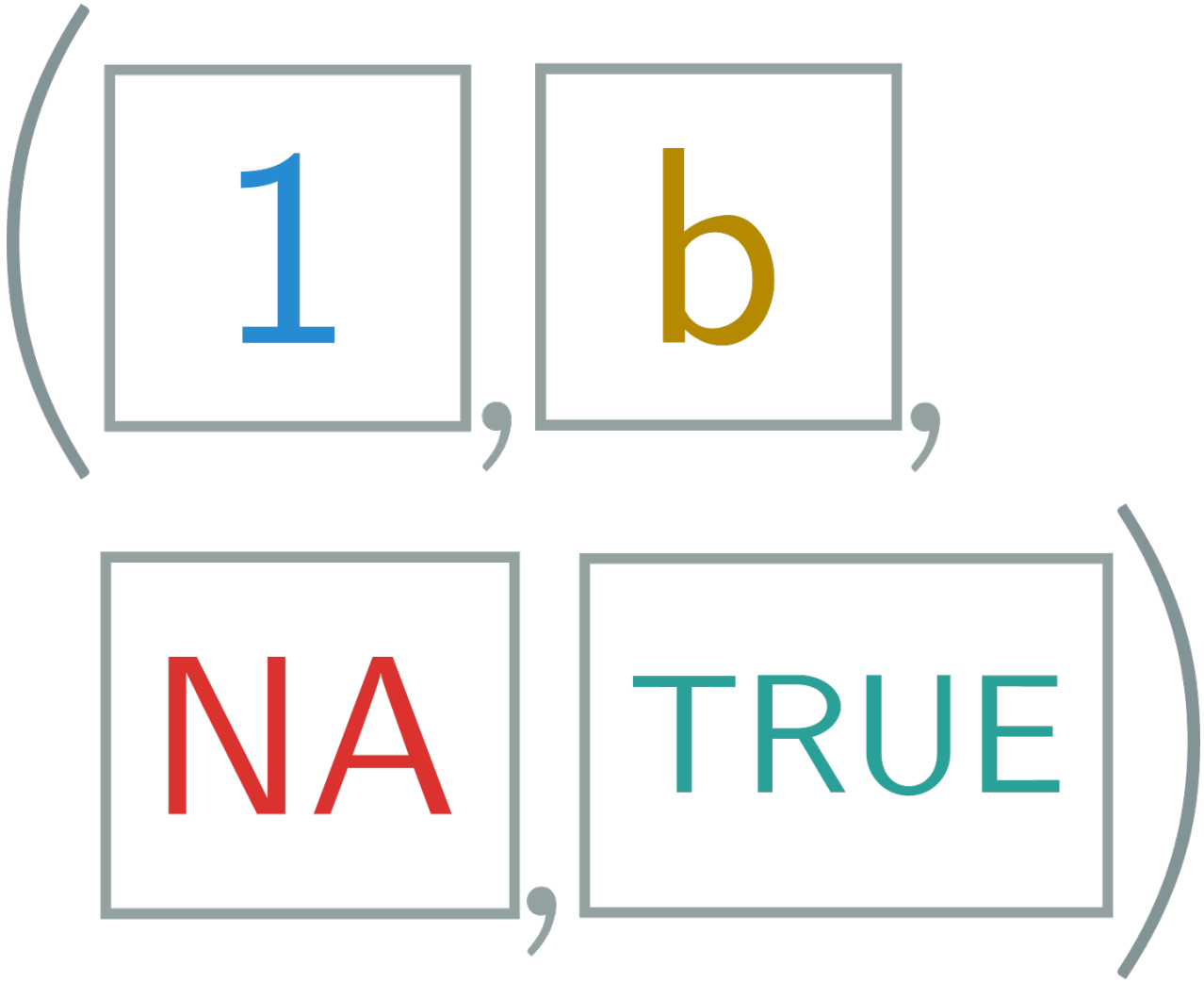
$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

They can be created with the `matrix` function.

```
> x <- 1:20
> matrix(x, ncol = 5)
     [,1] [,2] [,3] [,4] [,5]
[1,]    1    5    9   13   17
[2,]    2    6   10   14   18
[3,]    3    7   11   15   19
[4,]    4    8   12   16   20
```

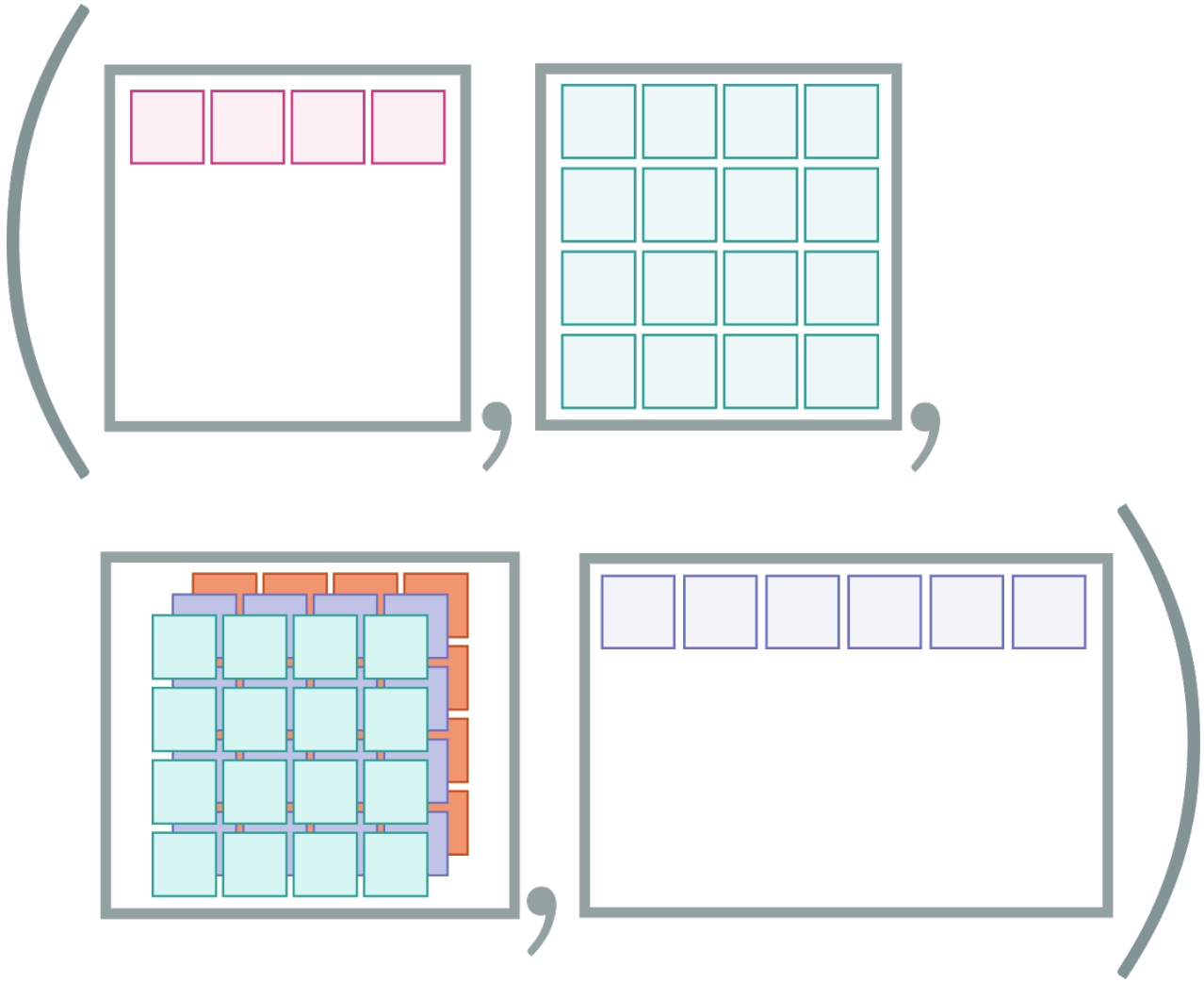
Lists

A list can contain objects of different types (including other lists).



Lists

A list can contain objects of different types (including other lists).



Defining a list

```r
my_list_unnamed <- list(
  12.1,
  "Tuesday",
  c(TRUE, FALSE, TRUE),
  matrix(1:20, ncol = 4)
)

# Same idea, but with names:
my_list_named <- list(
  number = 12.1,
  day = "Tuesday",
  flags = c(TRUE, FALSE, TRUE),
  mat = matrix(1:20, ncol = 4)
)
```

We'll cover how to select elements (e.g., `number`) from an object shortly.